

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Ingeniería en Sistemas de Información y Ciencias de la Computación

Investigación:
CSS Avanzado: Flexbox y CSS Grid

Estudiante: Jason Daniel Pivaral Miner

Carné: 1590-23-14836

Facultad: Ingeniería en Sistemas

Carrera: Ingeniería en Sistemas de Información
y Ciencias de la Computación

Curso: Desarrollo Web / Diseño de Interfaces

Fecha: 8 de agosto de 2026

Guatemala, C.A.

Índice

1. Introducción	3
2. CSS Flexbox	3
2.1. Definición y propósito	3
2.2. Conceptos fundamentales	3
2.3. Propiedades del contenedor flex	4
2.3.1. display	4
2.3.2. flex-direction	4
2.3.3. flex-wrap	5
2.3.4. flex-flow	5
2.3.5. justify-content	5
2.3.6. align-items	6
2.3.7. align-content	6
2.3.8. gap	6
2.4. Propiedades de los ítems flex	7
2.4.1. order	7
2.4.2. flex-grow	7
2.4.3. flex-shrink	7
2.4.4. flex-basis	8
2.4.5. flex (shorthand)	8
2.4.6. align-self	9
2.5. Ejemplo práctico: barra de navegación con Flexbox	9
2.6. Resumen de propiedades Flexbox	10
3. CSS Grid Layout	10
3.1. Definición y propósito	10
3.2. Conceptos fundamentales	11
3.3. Propiedades del contenedor Grid	11
3.3.1. display	12
3.3.2. grid-template-columns y grid-template-rows	12
3.3.3. grid-template-areas	12
3.3.4. gap	13
3.3.5. justify-items y align-items	13
3.3.6. justify-content y align-content	14

3.3.7.	grid-auto-flow	14
3.3.8.	grid-auto-columns y grid-auto-rows	15
3.4.	Propiedades de los ítems Grid	15
3.4.1.	grid-column y grid-row	15
3.4.2.	grid-area	15
3.4.3.	justify-self y align-self	16
3.5.	La función repeat() y la unidad fr	16
3.6.	Ejemplo práctico: layout de página completa con Grid	17
3.7.	Ejemplo: galería de imágenes responsive	18
3.8.	Resumen de propiedades Grid	19
4.	Comparación: Flexbox vs CSS Grid	20
4.1.	¿Cuándo usar cada uno?	20
5.	Compatibilidad y soporte de navegadores	21
6.	Buenas prácticas	21
7.	Conclusiones	21

Introducción

El diseño de interfaces web ha evolucionado de forma significativa desde los primeros días de HTML. Durante años, los desarrolladores dependieron de técnicas basadas en tablas, posicionamiento flotante (`float`) y posicionamiento absoluto para construir maquetaciones. Estas técnicas resultaban complejas, difíciles de mantener y poco intuitivas.

Con la llegada de CSS3, el consorcio W3C introdujo dos módulos de maquetación que revolucionaron el trabajo front-end: **Flexbox** (Flexible Box Layout) y **CSS Grid Layout**. Ambas tecnologías resuelven el problema del posicionamiento de elementos en el plano bidimensional, pero cada una con enfoques, fortalezas y casos de uso distintos.

La presente investigación tiene como objetivo describir, analizar y comparar ambos sistemas de maquetación, abordando su sintaxis, propiedades, comportamiento y buenas prácticas de uso, con ejemplos de código representativos.

CSS Flexbox

Definición y propósito

Flexbox (Flexible Box Layout Module) es un modelo de maquetación unidimensional introducido en CSS3. Se dice que es unidimensional porque organiza los elementos a lo largo de *un solo eje* a la vez: horizontal (fila) o vertical (columna). Su principal propósito es distribuir espacio entre elementos de un contenedor de manera eficiente, incluso cuando su tamaño es desconocido o dinámico.

Flexbox es especialmente útil para:

- Centrar elementos vertical y horizontalmente.
- Distribuir espacio entre ítems de navegación.
- Alinear componentes de una barra de herramientas.
- Crear diseños de una sola dimensión (filas o columnas) con comportamiento flexible.

Conceptos fundamentales

Flexbox introduce una terminología específica que es necesario comprender antes de estudiar sus propiedades:

Contenedor flex (*flex container*)

El elemento padre sobre el que se aplica `display: flex` o `display: inline-flex`.

Ítems flex (*flex items*)

Los hijos directos del contenedor flex.

Eje principal (*main axis*)

El eje sobre el que se colocan los ítems. Por defecto es horizontal (de izquierda a derecha).

Eje cruzado (*cross axis*)

El eje perpendicular al principal. Por defecto es vertical.

Main start / Main end

Puntos de inicio y fin del eje principal.

Cross start / Cross end

Puntos de inicio y fin del eje cruzado.

Propiedades del contenedor flex**display**

La propiedad `display` activa el contexto flex:

```
1 .contenedor {  
2   display: flex;           /* bloque flex */  
3   /* display: inline-flex; */ /* en línea */  
4 }
```

Listing 1: Activar Flexbox

flex-direction

Define la dirección del eje principal:

```
1 .contenedor {  
2   flex-direction: row;      /* defecto: izq a der */  
3   /* flex-direction: row-reverse;   der a izq */  
4   /* flex-direction: column;        arriba a abajo */  
5   /* flex-direction: column-reverse; abajo a arriba */
```

```
6 }
```

Listing 2: flex-direction

flex-wrap

Controla si los ítems se envuelven a una nueva línea cuando no caben:

```
1 .contenedor {  
2   flex-wrap: nowrap;    /* defecto: no envuelva */  
3   /* flex-wrap: wrap;    envuelva hacia abajo */  
4   /* flex-wrap: wrap-reverse; envuelva hacia arriba */  
5 }
```

Listing 3: flex-wrap

flex-flow

Atajo para combinar flex-direction y flex-wrap:

```
1 .contenedor {  
2   flex-flow: row wrap;  
3 }
```

Listing 4: flex-flow (shorthand)

justify-content

Alinea los ítems a lo largo del **eje principal**:

```
1 .contenedor {  
2   justify-content: flex-start;    /* defecto */  
3   /* justify-content: flex-end;    */  
4   /* justify-content: center;      */  
5   /* justify-content: space-between; */  
6   /* justify-content: space-around; */  
7   /* justify-content: space-evenly; */  
8 }
```

Listing 5: justify-content

align-items

Alinea los ítems a lo largo del **eje cruzado** (en una sola línea):

```
1 .contenedor {  
2   align-items: stretch;      /* defecto: estira al alto del  
   contenedor */  
3   /* align-items: flex-start; */  
4   /* align-items: flex-end;   */  
5   /* align-items: center;     */  
6   /* align-items: baseline;   */  
7 }
```

Listing 6: align-items

align-content

Alinea las líneas de ítems cuando hay múltiples filas (requiere **flex-wrap: wrap**):

```
1 .contenedor {  
2   align-content: flex-start;  
3   /* align-content: flex-end;      */  
4   /* align-content: center;        */  
5   /* align-content: space-between; */  
6   /* align-content: space-around;  */  
7   /* align-content: stretch;      */  
8 }
```

Listing 7: align-content

gap

Define el espacio entre ítems (también acepta **row-gap** y **column-gap** por separado):

```
1 .contenedor {  
2   gap: 16px;      /* mismo espacio en ambos ejes */  
3   /* gap: 10px 20px; fila columna */  
4   /* row-gap: 10px;   */  
5   /* column-gap: 20px; */  
6 }
```

Listing 8: gap en Flexbox

Propiedades de los ítems flex

order

Cambia el orden visual de un ítem sin modificar el HTML:

```
1 .item-especial {  
2   order: -1;    /* aparece primero aunque este despues en el HTML */  
3 }
```

Listing 9: order

flex-grow

Define cuánto puede crecer un ítem relativo a los demás cuando hay espacio disponible:

```
1 .item {  
2   flex-grow: 0;    /* defecto: no crece */  
3 }  
4  
5 .item-expandible {  
6   flex-grow: 1;    /* ocupa el espacio sobrante */  
7 }  
8  
9 .item-doble {  
10  flex-grow: 2;    /* crece el doble que uno con flex-grow: 1 */  
11 }
```

Listing 10: flex-grow

flex-shrink

Define cuánto puede encogerse un ítem cuando hay poco espacio:


```
1 .item {
2   flex-shrink: 1;    /* defecto: puede encoger */
3 }
4
5 .item-figo {
6   flex-shrink: 0;    /* no se encoge */
7 }
```

Listing 11: flex-shrink

flex-basis

Define el tamaño base de un ítem antes de que se distribuya el espacio sobrante:

```
1 .item {
2   flex-basis: auto;    /* defecto: usa el width/height del
3     elemento */
4   /* flex-basis: 200px; */
5   /* flex-basis: 30%; */
6   /* flex-basis: 0;     empezar sin tamaño base */
7 }
```

Listing 12: flex-basis

flex (shorthand)

Atajo que combina flex-grow, flex-shrink y flex-basis:

```
1 .item {
2   flex: 0 1 auto;    /* defecto */
3   /* flex: 1;         equivale a flex: 1 1 0 */
4   /* flex: auto;      equivale a flex: 1 1 auto */
5   /* flex: none;      equivale a flex: 0 0 auto */
6 }
```

Listing 13: flex (shorthand)

align-self

Sobreescribe `align-items` para un ítem individual:

```
1 .item-especial {
2   align-self: flex-end;
3   /* align-self: flex-start; */
4   /* align-self: center;      */
5   /* align-self: stretch;    */
6   /* align-self: auto;        hereda del contenedor */
7 }
```

Listing 14: align-self

Ejemplo práctico: barra de navegación con Flexbox

```
1  /* HTML asumido:
2    <nav class="navbar">
3      <div class="logo">Logo </div>
4      <ul class="nav-links">
5        <li><a href="#">Inicio </a></li>
6        <li><a href="#">Acerca </a></li>
7        <li><a href="#">Contacto </a></li>
8      </ul>
9      <button class="cta">Ingresar </button>
10    </nav>
11  */
12
13 .navbar {
14   display: flex;
15   justify-content: space-between;
16   align-items: center;
17   padding: 0 2rem;
18   height: 60px;
19   background-color: #003875;
20 }
21
22 .nav-links {
23   display: flex;
```

```

24  list-style: none;
25  gap: 1.5rem;
26  margin: 0;
27  padding: 0;
28  }
29
30  .cta {
31    flex-shrink: 0;
32  }

```

Listing 15: Navegación responsive con Flexbox

Resumen de propiedades Flexbox

Cuadro 1: Propiedades Flexbox — Contenedor e Ítems

Propiedad	Aplica a	Descripción
<code>display: flex</code>	Contenedor	Activa el contexto flex
<code>flex-direction</code>	Contenedor	Dirección del eje principal
<code>flex-wrap</code>	Contenedor	Envolvimiento de ítems
<code>justify-content</code>	Contenedor	Alineación en eje principal
<code>align-items</code>	Contenedor	Alineación en eje cruzado (una línea)
<code>align-content</code>	Contenedor	Alineación en eje cruzado (multi-línea)
<code>gap</code>	Contenedor	Espacio entre ítems
<code>order</code>	Ítem	Orden visual
<code>flex-grow</code>	Ítem	Factor de crecimiento
<code>flex-shrink</code>	Ítem	Factor de encogimiento
<code>flex-basis</code>	Ítem	Tamaño base
<code>flex</code>	Ítem	Shorthand grow/shrink/basis
<code>align-self</code>	Ítem	Alineación individual

CSS Grid Layout

Definición y propósito

CSS Grid Layout (o simplemente CSS Grid) es un sistema de maquetación *bidimensional* introducido como estándar CSS Level 1 en 2017. A diferencia de Flexbox, Grid permite

controlar simultáneamente filas **y** columnas, lo que lo convierte en la herramienta más poderosa para estructurar el layout general de una página.

CSS Grid es especialmente adecuado para:

- Definir la estructura principal de una página (header, sidebar, main, footer).
- Crear galerías de imágenes con posicionamiento preciso.
- Superponer elementos en la misma celda.
- Diseños complejos que requieren control en ambos ejes al mismo tiempo.

Nota

CSS Grid no es un reemplazo de Flexbox. Ambos coexisten y se complementan: Grid para el layout macro de la página, Flexbox para alinear elementos dentro de cada componente.

Conceptos fundamentales

Grid Container

El elemento padre con `display: grid`.

Grid Item

Los hijos directos del grid container.

Grid Line

Las líneas divisorias que forman la estructura. Se numeran de 1 en adelante (también desde -1 hacia atrás).

Grid Track

El espacio entre dos líneas adyacentes (una fila o columna).

Grid Cell

La intersección de una fila y una columna; la unidad más pequeña del grid.

Grid Area

Un rectángulo de una o más celdas nombrado explícitamente.

Unidad fr

Fracción del espacio disponible tras restar los tamaños fijos.

Propiedades del contenedor Grid

display

```
1 .contenedor {  
2   display: grid;  
3   /* display: inline-grid; */  
4 }
```

Listing 16: Activar CSS Grid

grid-template-columns y grid-template-rows

Definen el número y tamaño de las columnas y filas explícitas:

```
1 .contenedor {  
2   /* 3 columnas iguales usando la unidad fr */  
3   grid-template-columns: 1fr 1fr 1fr;  
4  
5   /* Equivalente usando repeat() */  
6   grid-template-columns: repeat(3, 1fr);  
7  
8   /* Columnas mixtas: fija, flexible, fija */  
9   grid-template-columns: 200px 1fr 100px;  
10  
11  /* Filas con alturas definidas */  
12  grid-template-rows: 80px auto 60px;  
13  
14  /* minmax: columnas que no sean menores a 200px ni mayores a 1fr  
   */  
15  grid-template-columns: repeat(3, minmax(200px, 1fr));  
16 }
```

Listing 17: Definir columnas y filas

grid-template-areas

Permite nombrar regiones del grid usando texto, facilitando la legibilidad:

```
1 .contenedor {  
2   display: grid;
```

```
3  grid-template-columns: 220px 1fr;
4  grid-template-rows: 70px 1fr 50px;
5  grid-template-areas:
6      "header  header"
7      "sidebar main"
8      "footer  footer";
9  }
10
11 .header { grid-area: header; }
12 .sidebar { grid-area: sidebar; }
13 .main { grid-area: main; }
14 .footer { grid-area: footer; }
```

Listing 18: grid-template-areas

gap

Igual que en Flexbox, define el espacio entre filas y columnas:

```
1  .contenedor {
2      gap: 20px;                /* igual en filas y columnas */
3      /* gap: 16px 24px;  fila columna */
4      /* row-gap: 16px;   */
5      /* column-gap: 24px; */
6  }
```

Listing 19: gap en Grid

justify-items y align-items

Alinean el contenido de **todas las celdas** del grid:

```
1  .contenedor {
2      /* Alinear horizontalmente dentro de cada celda */
3      justify-items: start;    /* start | end | center | stretch */
4
5      /* Alinear verticalmente dentro de cada celda */
6      align-items: center;     /* start | end | center | stretch */
7  }
```

```
8  /* Shorthand */
9  place-items: center start;
10 }
```

Listing 20: Alineación de celdas

justify-content y align-content

Alinean el **grid completo** dentro del contenedor cuando hay espacio sobrante:

```
1  .contenedor {
2    justify-content: center;
3    /* justify-content: start | end | space-between | space-around |
       space-evenly */
4
5    align-content: start;
6    /* align-content: center | end | space-between | stretch */
7
8    /* Shorthand */
9    place-content: center;
10 }
```

Listing 21: Alineación del grid completo

grid-auto-flow

Controla cómo se colocan los ítems que no tienen posición explícita:

```
1  .contenedor {
2    grid-auto-flow: row;      /* defecto: llena fila por fila */
3    /* grid-auto-flow: column;  llena columna por columna */
4    /* grid-auto-flow: dense;    rellena huecos con items pequenos */
5    /* grid-auto-flow: row dense; */
6  }
```

Listing 22: grid-auto-flow

grid-auto-columns y grid-auto-rows

Definen el tamaño de las pistas implícitas (creadas automáticamente):

```
1 .contenedor {  
2   grid-auto-rows: 150px;           /* filas implícitas de 150px */  
3   /* grid-auto-rows: minmax(100px, auto); */  
4   grid-auto-columns: 1fr;  
5 }
```

Listing 23: Pistas implícitas

Propiedades de los ítems Grid

grid-column y grid-row

Ubican un ítem indicando entre qué líneas del grid debe ocupar espacio:

```
1 .item {  
2   /* Ocupa desde la línea 1 hasta la línea 3 en columnas */  
3   grid-column: 1 / 3;  
4  
5   /* Alternativa con span: "desde donde esta, abarca 2 columnas" */  
6   grid-column: span 2;  
7  
8   /* Ocupa la segunda fila completa */  
9   grid-row: 2 / 3;  
10  
11  /* Desde línea 1 hasta el final (-1) */  
12  grid-column: 1 / -1;  
13 }
```

Listing 24: Posicionamiento con líneas

grid-area

Asigna a un ítem una área nombrada, o define su posición con shorthand:

```
1 /* Con nombre (requiere grid-template-areas en el contenedor) */  
2 .header {
```



```
3  grid-area: header;
4  }
5
6  /* Shorthand: row-start / col-start / row-end / col-end */
7  .item {
8  grid-area: 1 / 1 / 3 / 4;
9  }
```

Listing 25: grid-area

justify-self y align-self

Sobreescriben la alineación para un ítem específico:

```
1  .item-especial {
2  justify-self: end;
3  align-self: center;
4  /* place-self: center end; shorthand */
5  }
```

Listing 26: Alineación individual

La función repeat() y la unidad fr

```
1  .contenedor {
2  /* 4 columnas de igual tamaño */
3  grid-template-columns: repeat(4, 1fr);
4
5  /* auto-fill: crea todas las columnas que quepan >= 200px */
6  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
7
8  /* auto-fit: igual pero colapsa columnas vacías */
9  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
10 }
```

Listing 27: Uso avanzado de repeat() y fr

auto-fill vs auto-fit

auto-fill crea columnas vacías para llenar el contenedor. **auto-fit** colapsa esas columnas vacías y estira las existentes para que llenen el espacio. En la práctica, **auto-fit** es más común cuando se quiere que el contenido ocupe todo el ancho.

Ejemplo práctico: layout de página completa con Grid

```
1  /* HTML asumido:
2      <div class="pagina">
3          <header class="header">Header</header>
4          <nav class="sidebar">Sidebar</nav>
5          <main class="contenido">Contenido</main>
6          <aside class="panel">Panel</aside>
7          <footer class="footer">Footer</footer>
8      </div>
9  */
10
11 .pagina {
12     display: grid;
13     grid-template-columns: 220px 1fr 200px;
14     grid-template-rows: auto 1fr auto;
15     grid-template-areas:
16         "header    header    header"
17         "sidebar   contenido  panel"
18         "footer    footer    footer";
19     min-height: 100vh;
20     gap: 12px;
21 }
22
23 .header { grid-area: header;    background: #003875; color:
24           white; padding: 1rem; }
25 .sidebar { grid-area: sidebar;   background: #f0f4f8; padding:
26           1rem; }
27 .contenido{ grid-area: contenido; padding: 1rem; }
28 .panel    { grid-area: panel;    background: #f0f4f8; padding:
29           1rem; }
30 .footer   { grid-area: footer;   background: #333; color: white;
31           padding: 1rem; }
```

Listing 28: Layout completo de página con CSS Grid

Ejemplo: galería de imágenes responsive

```
1 .galeria {  
2   display: grid;  
3   grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));  
4   gap: 16px;  
5   padding: 16px;  
6 }  
7  
8 .galeria img {  
9   width: 100%;  
10  height: 180px;  
11  object-fit: cover;  
12  border-radius: 8px;  
13 }  
14  
15 /* Destacar la primera imagen: ocupa 2x2 */  
16 .galeria img:first-child {  
17   grid-column: span 2;  
18   grid-row: span 2;  
19 }
```

Listing 29: Galería responsive con auto-fit y minmax

Resumen de propiedades Grid

Cuadro 2: Propiedades CSS Grid — Contenedor e Ítems

Propiedad	Aplica a	Descripción
<code>display: grid</code>	Contenedor	Activa el contexto grid
<code>grid-template-columns</code>	Contenedor	Define columnas explícitas
<code>grid-template-rows</code>	Contenedor	Define filas explícitas
<code>grid-template-areas</code>	Contenedor	Nombra regiones del grid
<code>gap</code>	Contenedor	Espacio entre celdas
<code>justify-items</code>	Contenedor	Alineación horizontal de celdas
<code>align-items</code>	Contenedor	Alineación vertical de celdas
<code>justify-content</code>	Contenedor	Posición horizontal del grid
<code>align-content</code>	Contenedor	Posición vertical del grid
<code>grid-auto-flow</code>	Contenedor	Flujo de ítems implícitos
<code>grid-auto-rows/columns</code>	Contenedor	Tamaño de pistas implícitas
<code>grid-column</code>	Ítem	Posición/extensión en columnas
<code>grid-row</code>	Ítem	Posición/extensión en filas
<code>grid-area</code>	Ítem	Área nombrada o posición completa
<code>justify-self</code>	Ítem	Alineación horizontal individual
<code>align-self</code>	Ítem	Alineación vertical individual

Comparación: Flexbox vs CSS Grid

Cuadro 3: Flexbox vs CSS Grid — Comparación directa

Característica	Flexbox	CSS Grid
Dimensionalidad	Unidimensional (fila O columna)	Bidimensional (filas Y columnas)
Caso de uso ideal	Componentes y alineación interna	Layout general de la página
Control de ejes	Solo un eje a la vez	Ambos ejes simultáneamente
Posicionamiento	Flujo natural + orden	Posición explícita por líneas o áreas
Soporte implícito	Sí, crea filas/columnas automáticamente	Sí, con <code>grid-auto-*</code>
Responsive sin media queries	Limitado	<code>auto-fit</code> + <code>minmax</code> muy poderoso
Superposición de ítems	No nativo	Sí, con líneas explícitas
Compatibilidad navegadores	Excelente (IE10+)	Muy buena (Edge 16+, sin IE)
Curva de aprendizaje	Moderada	Alta

¿Cuándo usar cada uno?

Usar Flexbox cuando:

- Se necesita alinear elementos en una sola dirección.
- Los ítems tienen tamaños variables y se deben adaptar dinámicamente.
- Se trabaja con componentes pequeños: botones, navbars, formularios.
- No se conoce de antemano la cantidad de ítems.

Usar CSS Grid cuando:

- Se diseña el layout macro de la página (header, aside, main, footer).
- Se necesita posicionar ítems en filas y columnas al mismo tiempo.
- Se requiere superposición de elementos.
- Se quiere crear una cuadrícula responsive sin media queries complejas.

Compatibilidad y soporte de navegadores

Actualmente ambas tecnologías cuentan con soporte amplio en los navegadores modernos:

Cuadro 4: Soporte en navegadores (2024)

Tecnología	Chrome	Firefox	Safari	Edge
Flexbox	29+	28+	9+	12+
CSS Grid	57+	52+	10.1+	16+

Para verificar compatibilidad actualizada se recomienda consultar [Can I Use](https://caniuse.com) (caniuse.com).

Buenas prácticas

1. **Combinar Grid y Flexbox:** no son excluyentes. Usar Grid para el layout de la página y Flexbox dentro de cada componente es el patrón más eficiente.
2. **Preferir gap sobre márgenes:** desde CSS3, gap funciona en Grid y Flexbox, simplificando el control del espaciado.
3. **Diseño mobile-first:** definir la estructura base para móvil y ampliarla con @media queries.
4. **Usar minmax() y auto-fit:** para grids responsive sin media queries adicionales.
5. **Evitar valores de posición rígidos:** preferir unidades relativas (`fr`, `%`, `em`) sobre píxeles fijos.
6. **Nombrar las áreas del grid:** usar `grid-template-areas` mejora la legibilidad del código y facilita el mantenimiento.
7. **No abusar de order:** alterar el orden visual puede perjudicar la accesibilidad (los lectores de pantalla siguen el orden del DOM).

Conclusiones

CSS Flexbox y CSS Grid representan los dos pilares modernos del diseño web con hojas de estilo. Flexbox resuelve de forma elegante la alineación de elementos en un solo eje, resultando ideal para componentes de interfaz. Grid, por su parte, proporciona control

total sobre el plano bidimensional, siendo la herramienta definitiva para estructurar layouts complejos.

La comprensión profunda de ambas tecnologías es indispensable para cualquier desarrollador web moderno, ya que permiten construir interfaces coherentes, responsivas y mantenibles sin depender de frameworks o librerías externas de terceros.

La tendencia actual en el desarrollo front-end apunta a una adopción creciente de CSS Grid para estructuras macro y Flexbox para componentes internos, combinados con el uso de propiedades modernas como `gap`, `clamp()`, `min()`, `max()`, `container queries` y `subgrid`, estas últimas marcando el siguiente paso en la evolución del CSS Layout.

Referencias

- W3C. (2018). *CSS Flexible Box Layout Module Level 1*. <https://www.w3.org/TR/css-flexbox-1/>
- W3C. (2020). *CSS Grid Layout Module Level 1*. <https://www.w3.org/TR/css-grid-1/>
- Mozilla Developer Network (MDN). (2024). *CSS Flexbox*. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout
- Mozilla Developer Network (MDN). (2024). *CSS Grid Layout*. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout
- Coyier, C. (2022). *A Complete Guide to Flexbox*. CSS-Tricks. <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- Coyier, C. (2022). *A Complete Guide to CSS Grid*. CSS-Tricks. <https://css-tricks.com/snippets/css/complete-guide-grid/>
- Can I Use. (2024). <https://caniuse.com>